API Design guidelines and recommended practices

The purpose of this document is to provide some important design aspects and recommended practices for implementing APIs

API Design Considerations

- **Coupling**
  - API based systems typically has multiple APIs and underlying subsystems involved in the solution. Low coupling will allow you to change or upgrade the components independently, without this impacting any other components in the system.

    By focusing on building the components as atomic, self-contained parts, you can get optimal flexibility and improve your reusability
- **Chattiness**
  - Chattiness refers to the number of calls the API consumer is required to make to get the necessary information. Chattiness must be avoided in your API design. Otherwise, it will lead to very poor end user experience. Multiple API calls consume both network bandwidth and increase the overall transaction time for any scenario.

    API operations must be designed in a way that they provide the information or data that's been requested in one go. Transaction modeling techniques that use sequence diagrams can be adopted to identify whether there is chattiness in the workflow.

    Chattiness is also considered to be a performance anti-pattern. Refer to this article to find out more: Chatty I/O antipattern - Azure Architecture Center
- **Client Complexity**
  - Please refer to BFF pattern section below
- **Cognitive Complexity**
  - The more code or logic that you have in your API, the more difficult it will be to unit test all the paths, so any upgrades or changes are risky, In general, cognitive complexity must be low for a good API design. If there are multiple process steps, you should remodel your business workflow to avoid incurring high cognitive complexity in your components.
  - SonarQube has been integrated with CI/CD pipelines for all the major projects across Core Engineering applications. SonarQube can measure the Cognitive Complexity of your code
- **Caching**

- Please refer to 'Improving API Performance' under API Management Strategy and Best Practices

- **Response Caching**
  - Please refer to 'Improving API Performance' under API Management Strategy and Best Practices
- **Discoverability**
  - Discoverability refers to the ease with which information about the list of APIs, their corresponding operations, and usage techniques can be accessed. The adoption of the API platform is largely dependent on how easily developers can complete their integrations. Hence, you must provide as much information as possible in the definition files to avoid any ambiguity in understanding.
- **Versioning**
  - APIs typically undergo multiple revisions as part of the change management cycles. Hence, it is imperative that you adopt a proper versioning strategy to distinctly identify what changed in which version. Furthermore, the changes must be backward compatible to ensure that any existing client integrations are not impacted.
  - for more details, please refer to link below
    - Versioning for API Endpoints
- **Security**
  - Please refer to 'Secure by Design' section under API Implementation - Recommended Practices

API Implementation - Recommended Practices

- **Sequence Diagrams -** It is recommended to use sequence diagram as part of your low-level design to depict request-response flows. It aids in capturing all the subsystems that will be involved as part of complete operation
- **Embrace Clean Architecture (Hexagonal Architecture)**
  - Separation of Core application logic from external dependencies
  - Database connections, user interface, message handlers all connected through set of adapters/ports to the core application.
- **Secure by Design**
  - Implement Authorization -
    - Role based access and protect methods by Okta
    - protect against CSRF attacks
  - Input Validation - Url Validation, Validate incoming contents types, validate response types

- Output encoding - Security headers, JSON /XML encoding
- Cryptography-
  - Data in transit: Mandate the use of TLS,
  - Data in storage: When handling sensitive data, ensure that it is encrypted using modern and secure cryptography techniques.
  - Message integrity: Make use of OAuth tokens in the request header to ensure the integrity of the transmission
- Avoid keeping secrets in code and utilize secure credential management for your deployment environment.

- **Size and Granularity**
  - xx
- **User-digestible response codes and Messages**
  - xx
- **Backends for Front-ends**
  - xxx
- **Use cloud design patterns**
  - Azure Architecture Center has published a collection of cloud design patterns that development teams must plan to incorporate in their designs.

    You can review the full catalog of cloud design patterns here: Cloud Design Patterns - Azure Architecture Center .

    These patterns are mostly structural and behavioral patterns that address common challenges for cloud-based applications.

API Management Strategy and Best Practices

For API Management and security considerations, please refer to link below

APIs and API Management

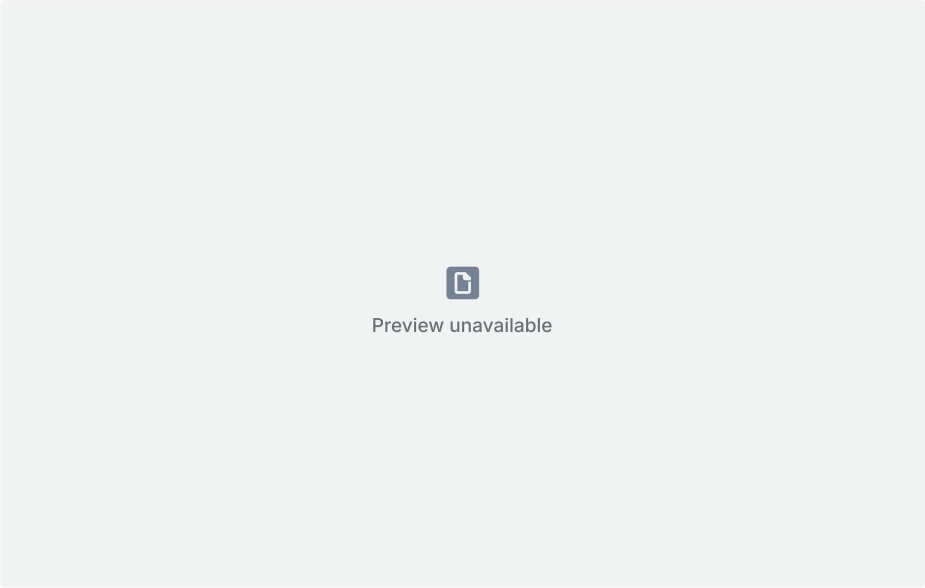
- **Policies for internal and external consumptions of APIs**
  - Azure APIM sits between your API clients and your backend APIs, and it will add a few useful features to your backend APIs. APIM consists of three main components, Azure portal, Azure API Gateway, and Developer portal. APIM policies will be configured in the Azure portal, and they execute inside Azure API Gateway
  - Policies are a powerful capability of the system that allow the publisher to change the behavior of the API through configuration. Policies are a collection of Statements that are executed sequentially on the request or response of an API. Popular Statements include

format conversion from XML to JSON and call rate limiting to restrict the amount of incoming calls from a developer. Many more policies are available out of the box
  - Please refer to link below on policies and defining policies.
  - Azure API Management policy reference
- **Improving API Performance**
  - APIs and operations in API Management can be configured with response caching. Response caching can significantly reduce latency for API callers and backend load for API providers
  - Please refer to link below on APIM caching policies
    - Azure API Management policy reference
    - Custom Caching in Azure API Management
- **APIM Logging and Monitoring**
  - With Azure Monitor, you can visualize, query, route, archive, and take actions on the metrics or logs coming from your Azure API Management service
  - Tutorial - Monitor APIs in Azure API Management
- **Integrating APIM with Azure Application Gateway**
  - You can configure Azure API Management in a virtual network in internal mode, which makes it accessible only within the virtual network. Azure Application Gateway is a platform as a service (PaaS) that acts as a Layer-7 load balancer. It acts as a reverse-proxy service and provides among its offerings Azure Web Application Firewall (WAF).

    By combining API Management provisioned in an internal virtual network with the Application Gateway front end, you can:
    - Use the same API Management resource for consumption by both internal consumers and external consumers.
    - Use a single API Management resource and have a subset of APIs defined in API Management available for external consumers.
    - Provide a turnkey way to switch access to API Management from the public internet on and off
    - for more details please refer to link below
      - Use API Management in a virtual network with Azure Application Gateway - Azure API Management
- **Integrating APIM with AKS**
  - xxx

AKS Best Practices - A quick check-list

- **Refrain from injecting sensitive information into images and use Secrets instead**
  - always use Secrets—either Kubernetes Secrets or Azure Key Vaults—for sensitive information
- **Implement Pod identity**
  - Don't have fixed credentials stored in Pod images. Rather you can use Pod identities, which use the Azure Identity solution for all the access to the desired (Azure) resources. These credentials could be any credentials used to talk with other Azure services, like Azure SQL or Azure Storage



A developer created a Pod that uses a managed identity to request access to an Azure SQL database Image source: Microsoft Documentation

- **Use a Kubernetes Namespace**.
  - Namespaces are the logical partitions of your resources. They not only enforce the separation of resources but also limit the permissible user scope
  - Avoid using the default namespace.
- **Specify the correct security context for a pod**
  - This is an important factor in deciding your Pod access control settings. If the context is not set, the Pod gets the default one, which exposes it with more rights.
- **Enforcement of compliance on the build image**
  - You must go through Azure Policy built-in definitions for Azure Kubernetes Service; see `https://docs.microsoft.com/en-us/azure/aks/policy-reference?ref=akschecklist`

References: